

QUERYQUEST ASSIGNMENT – 02

PART I:

Q1. The query returns 0 because NULL represents a missing or unknown value. In SQL, NULL cannot be compared using = or != . Therefore, the condition phone_number = NULL is never true.

The correct query is:

```
SELECT COUNT(*)  
FROM Student  
WHERE phone_number IS NULL;
```

NULL is excluded from most aggregate functions and comparison operations because its value is unknown. Since SQL cannot determine whether an unknown value is equal to, greater than, or less than another value, it ignores NULLs in most calculations and comparisons.

Q2. Yes, the student is correct. The GROUP BY department clause already creates one row for each department. Since each department appears only once in the result, there are no duplicate rows for DISTINCT to remove. In this query, using DISTINCT does not change the output. The result would be the same even if DISTINCT were removed.

Q3. The difference in row count shows that some customers do not have matching records in the Order table. An INNER JOIN returns only customers who have placed orders, while a LEFT JOIN returns all customers, including those who have never placed an order. Therefore, if the LEFT JOIN returns more rows, it means there are customers without any orders.

Q4. This query is invalid because employee_name is neither included in the GROUP BY clause nor used in an aggregate function. When employees are grouped by department, there are many employee names within the same department. SQL would not know which employee name to display for that department. For example, if the Marketing department contains Ananya, Daya, and Maha, SQL cannot decide which name should appear alongside the average salary. There is no single correct value, hence standard SQL rejects the query.

CORRECT QUERY:

```
SELECT department, AVG(salary)  
FROM Employee  
GROUP BY department;
```

Q5. This query fails because aggregate functions like AVG() cannot be used in the WHERE clause. The WHERE clause is executed before the data is grouped, so the average salary has not been calculated yet. To filter groups based on an aggregate value, we use the HAVING clause, which is executed after GROUP BY.

QUERY EXECUTION ORDER:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT

Since WHERE comes before GROUP BY, it cannot use AVG(salary).

CORRECT QUERY:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. (a). A non-correlated subquery is more appropriate because the average product price needs to be calculated only once for the entire table.

```
SELECT *
FROM Product
WHERE price >
(
    SELECT AVG(price)
    FROM Product
);
```

(b). A correlated subquery is more appropriate because the average salary must be calculated separately for each employee's department.

```
SELECT *
FROM Employee e1
WHERE salary >
(
    SELECT AVG(salary)
    FROM Employee e2
    WHERE e1.department = e2.department
);
```

PERFORMANCE IMPLICATION:

A correlated subquery is generally slower because it may be executed once for every row in the outer query. When the outer table is large, this can significantly increase execution time. A non-correlated subquery is usually more efficient because it is executed only once.

PART II:

(a). SQL QUERY:

```
SELECT c.customer_id, c.name, c.email, c.phone
FROM Customer c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

EXPLANATION:

I used a **LEFT JOIN** because we need to include all customers, even those who have not placed any orders. Customers with no matching record in the Orders table will have NULL values for order details, so WHERE o.order_id IS NULL identifies customers who have never placed an order.

(b). SQL QUERY:

```
SELECT r.restaurant_id,
       r.name,
       AVG(order_value) AS average_order_value
FROM Restaurant r
JOIN (
  SELECT o.order_id,
         o.restaurant_id,
         SUM(m.price * oi.quantity) AS order_value
  FROM Orders o
  JOIN OrderItem oi
  ON o.order_id = oi.order_id
  JOIN MenuItem m
  ON oi.item_id = m.item_id
  GROUP BY o.order_id, o.restaurant_id
) AS OrderValues
ON r.restaurant_id = OrderValues.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(order_id) > 5;
```

EXPLANATION:

I used joins to combine data from the Orders, OrderItem, and MenuItem tables. The subquery calculates the value of each order by summing price × quantity and the outer query finds the average order value for each restaurant. The HAVING clause is used because we need to filter restaurants based on the number of orders after grouping.

(c). SQL QUERY:

```
SELECT restaurant_id, AVG(order_value) AS average_order_value
FROM (
    SELECT o.order_id,
           o.restaurant_id,
           SUM(m.price * oi.quantity) AS order_value
    FROM Orders o
    JOIN OrderItem oi
        ON o.order_id = oi.order_id
    JOIN MenuItem m
        ON oi.item_id = m.item_id
    GROUP BY o.order_id, o.restaurant_id
) AS OrderValues
GROUP BY restaurant_id
HAVING AVG(order_value) >
(
    SELECT AVG(order_value)
    FROM (
        SELECT o.order_id,
               SUM(m.price * oi.quantity) AS order_value
        FROM Orders o
        JOIN OrderItem oi
            ON o.order_id = oi.order_id
        JOIN MenuItem m
            ON oi.item_id = m.item_id
        GROUP BY o.order_id
    ) AS PlatformOrders
);
```

EXPLANATION:

I used a **non-correlated subquery** because the platform-wide average order value only needs to be calculated once. The outer query calculates the average order value for each restaurant, and the HAVING clause compares it with the overall platform average.

(d). SQL QUERY:

```

SELECT m.restaurant_id,
       m.item_name,
       SUM(oi.quantity) AS total_quantity
FROM MenuItem m
JOIN OrderItem oi
  ON m.item_id = oi.item_id
GROUP BY m.restaurant_id, m.item_id, m.item_name
HAVING SUM(oi.quantity) = (
  SELECT MAX(item_total)
  FROM (
    SELECT SUM(oi2.quantity) AS item_total
    FROM MenuItem m2
    JOIN OrderItem oi2
      ON m2.item_id = oi2.item_id
    WHERE m2.restaurant_id = m.restaurant_id
    GROUP BY m2.item_id
  ) AS ItemTotals
);

```

EXPLANATION:

I used joins to connect MenuItem and OrderItem and calculate the total quantity ordered for each menu item. A subquery is then used to find the maximum quantity ordered within each restaurant, and only the item(s) matching that maximum value are returned.

LIMITATION of GROUP BY + MAX():

A limitation of using GROUP BY and MAX() for top-N per group problems is that it does not automatically handle ties. If two menu items have the same highest quantity, additional logic is needed to decide whether both items should be returned or only one. For larger top-N problems, window functions such as RANK() or ROW_NUMBER() are often easier to use.

PART III:

Q1. Functional Dependencies:

1. student_id → student_name
2. course_id → course_name, instructor_id
3. instructor_id → instructor_name, instructor_office
4. (student_id, course_id) → grade, enrollment_date

Q2. Yes, the table is in 1NF because each column contains only one value and there are no repeating groups. Each row represents one student's enrollment in one course. Therefore, the table satisfies the requirements of First Normal Form (1NF).

Q3. The primary key is (student_id, course_id). The following are partial dependencies:

- student_id → student_name
- course_id → course_name, instructor_id

These violate 2NF because they depend on only part of the composite key instead of the whole key. This can cause update anomalies. For example, if a student's name changes, it must be updated in every row where that student appears.

Q4. The transitive dependency is:

- course_id → instructor_id
- instructor_id → instructor_name, instructor_office

This means that instructor_name and instructor_office depend on course_id through instructor_id. This violates 3NF because a non-key attribute is dependent on another non-key attribute. It can cause update anomalies. For example, if an instructor changes office, the office number would need to be updated in every row for all courses taught by that instructor.

Q5. The table can be decomposed into the following 3NF relations:

1. Student(student_id [PK], student_name)

- Based on: student_id → student_name

2. Instructor(instructor_id [PK], instructor_name, instructor_office)

- Based on: instructor_id → instructor_name, instructor_office

3. Course(course_id [PK], course_name, instructor_id)

- Based on: course_id → course_name, instructor_id

4. Enrollment(student_id [PK, FK], course_id [PK, FK], grade, enrollment_date)

- Based on: (student_id, course_id) → grade, enrollment_date

PART IV:

In Part II(a), the query for finding customers who have never placed an order could be written using either a LEFT JOIN or a subquery. I chose a LEFT JOIN because it

is easier to read and clearly shows that we want all customers, including those without matching orders. By checking for NULL values in the Orders table, it becomes simple to identify customers who have never placed an order.

In Part III, normalizing the Enrollment table into separate Student, Course, Instructor, and Enrollment tables reduces data redundancy and prevents update, insertion, and deletion anomalies. However, normalization can make queries more complex because information is spread across multiple tables and often requires joins to retrieve complete details. For example, getting a student's course and instructor information would require joining several tables. In real systems, some denormalization may be used to improve query performance and simplify reporting, even though it introduces some data redundancy.